AI MANUFACTURING QUALITY PLATFORM

Technical Reference — C++ + Python

Item	Detail
Open Standard	GigE Vision + GenICam (camera-agnostic)
Camera	Arducam UC852 / OV9782 — Color Global Shutter USB
Vision Layer	C++ 17 + OpenCV 4.x (real-time, < 10ms/frame)
App Layer	Python 3.10 + FastAPI (API, DB, dashboard, Claude)
AI	Claude API (Anthropic) via Claude Code terminal
DB	SQLite → PostgreSQL
MVP	8 Weeks

1. OPEN ARCHITECTURE REFERENCE

Base your design on these open standards. Each one is freely documented, camera-vendor-neutral, and used in production globally.

Standard	What It Defines	Why We Use It	Reference
GigE Vision 2.0	Network transport for industrial cameras	Camera-agnostic acquisition layer	visiononline.org/gige-vision
GenICam	Camera feature register map (exposure, gain, trigger)	Control any brand with same API	emva.org/standards/genicam
OpenCV 4.x	Computer vision algorithm library (C++ + Python)	Industry default for vision processing	opencv.org
ONVIF	IP camera interoperability profile	Future multi-camera network integration	onvif.org
REST / OpenAPI 3.0	API schema standard	ERP + dashboard integration contract	swagger.io/specification
MQTT / OPC-UA	Factory device messaging (PLC integration)	Future PLC ejector + sensor data	opcfoundation.org

2. LANGUAGE SPLIT — C++ vs PYTHON

This is not two separate projects. It is one system with the right language in each layer.

Layer	Language	Reason	Performance Target
Frame capture	C++	Direct USB/GigE DMA, zero-copy buffer	< 2ms per frame
Preprocessing	C++	OpenCV Mat operations, SIMD optimized	< 5ms per frame
Defect detection	C++	Canny, morphology, contours — CPU/GPU	< 15ms per frame
Decision engine	C++	Score aggregation, threshold compare	< 1ms
Result serialization	C++ → Python	JSON bridge via shared memory or pipe	< 1ms
API server	Python	FastAPI — developer speed > raw speed	< 100ms response
Database ORM	Python	SQLAlchemy — readable, safe	< 50ms query
Claude integration	Python	anthropic SDK is Python-native	< 10s report gen
Dashboard	Python + HTML	Jinja2 templates + Chart.js	WebSocket push

💡 Start with Python-only for Week 1–4 to move fast. Rewrite vision detectors in C++ in Week 5 when bottlenecks are measured. Never optimize early.

3. MODULE STRUCTURE

```
quality_platform/
├── cpp/                          # C++ vision layer
│   ├── include/
│   │   ├── capture.hpp           # Camera init, frame grab
│   │   ├── preprocessor.hpp      # Resize, denoise, normalize
│   │   └── detectors/
│   │       ├── base.hpp          # Abstract Detector interface
│   │       ├── surface.hpp       # Scratch / dust (Canny + morphology)
│   │       ├── color.hpp         # HSV anomaly
│   │       └── ensemble.hpp      # Combine scores → PASS/FAIL
│   ├── src/                      # .cpp implementations
│   ├── tests/                    # Google Test unit tests
│   └── CMakeLists.txt
│
├── src/                          # Python app layer
│   ├── vision_bridge.py          # Calls C++ binary, parses result JSON
│   ├── api/
│   │   ├── main.py               # FastAPI app + WebSocket
│   │   ├── routes/inspection.py  # POST /inspect  GET /results
│   │   └── schemas.py            # Pydantic models
│   ├── db/
│   │   ├── models.py             # SQLAlchemy tables
│   │   └── repository.py         # CRUD
│   ├── dashboard/index.html      # Live operator UI
│   └── utils/
│       ├── config.py             # Loads config.yaml
│       └── claude_helper.py      # Claude API: classify + report
│
├── tests/                        # Python integration + system tests
│   ├── unit/                     # Per-module pytest tests
│   ├── integration/              # API + DB tests
│   └── fixtures/                 # Sample images (PASS + FAIL)
│
├── claude_prompts/               # Saved prompts per module
├── config.yaml
├── requirements.txt
└── CMakeLists.txt                # Root build
```

4. C++ CORE — KEY INTERFACES

4.1 Abstract Detector Base

```
// cpp/include/detectors/base.hpp
#pragma once
#include <opencv2/opencv.hpp>
#include <vector>
#include <string>

struct DefectRegion {
    int x, y, w, h;       // bounding box
    float confidence;       // 0.0 - 1.0
    std::string type;       // 'scratch' | 'dust' | 'color' | 'brightness'
};

struct DetectionResult {
    float score;                        // 0.0 = bad, 1.0 = perfect
    std::vector<DefectRegion> defects;
    long long duration_ms;
};

class Detector {
public:
    virtual ~Detector() = default;
    virtual DetectionResult detect(const cv::Mat& frame) = 0;
    virtual std::string name() const = 0;
};
```

4.2 Ensemble — Combines All Detectors

```
// cpp/include/detectors/ensemble.hpp
#pragma once
#include 'base.hpp'
#include <map>

struct EnsembleConfig {
    std::map<std::string, float> weights;  // e.g. {surface:0.5, color:0.3, brightness:0.2}
    float pass_threshold;                   // default 0.7
};

struct FinalDecision {
    bool pass;
    float confidence;
    std::vector<DefectRegion> all_defects;
    std::string result_json();  // serialize to JSON for Python bridge
};

class Ensemble {
    std::vector<std::unique_ptr<Detector>> detectors_;
    EnsembleConfig config_;
public:
    Ensemble(EnsembleConfig cfg);
    void add(std::unique_ptr<Detector> d);
    FinalDecision run(const cv::Mat& frame);
};
```

4.3 Python ↔ C++ Bridge

```
# src/vision_bridge.py
import subprocess, json, cv2, numpy as np
```

```
def inspect_frame(frame: np.ndarray) -> dict:
    '''Call compiled C++ binary, return decision as dict.'''
    _, buf = cv2.imencode('.jpg', frame)
    result = subprocess.run(
        ['./cpp/build/inspector'],  # compiled C++ binary
        input=buf.tobytes(),
        capture_output=True
    )
    return json.loads(result.stdout)
    # Returns: {pass: bool, confidence: float, defects: [...], duration_ms: int}
```

5. 8-WEEK MVP PLAN

Week	Goal	Language	Test Gate
W1	Camera live, dev env setup	Python (cv2.VideoCapture)	Frame 1280×800 saved
W2	Python vision pipeline	Python + OpenCV	Preprocess < 10ms, unit tests pass
W3	Surface detector (Python)	Python	≥ 80% accuracy on 10 fixtures
W4	Color + brightness + ensemble	Python	All detectors, integration test pass
W5	C++ rewrite: capture + detectors	C++ 17 + OpenCV	Same accuracy, 3× faster than Python
W6	Decision + DB + FastAPI	C++ bridge + Python	Loop saves to SQLite, API 200 OK
W7	Live dashboard + Claude reports	Python + HTML	WebSocket < 1s, report generated
W8	ERP webhook + CI/CD + hardening	Python + GitHub Actions	8-hour soak, CI green

💡 Weeks 1–4 use Python-only so students can move fast. C++ rewrite in Week 5 only — after you have working tests to verify correctness is preserved.

6. TEST STRATEGY

6.1 C++ Tests — Google Test

```
// cpp/tests/test_surface_detector.cpp
#include <gtest/gtest.h>
#include 'detectors/surface.hpp'

TEST(SurfaceDetector, CleanImageNoDefects) {
    cv::Mat frame = cv::imread('fixtures/clean_metal_01.jpg');
    SurfaceDetector det;
    auto result = det.detect(frame);
    EXPECT_TRUE(result.defects.empty());
    EXPECT_GE(result.score, 0.8f);
}

TEST(SurfaceDetector, ScratchDetected) {
    cv::Mat frame = cv::imread('fixtures/scratch_01.jpg');
    SurfaceDetector det;
    auto result = det.detect(frame);
    EXPECT_FALSE(result.defects.empty());
    EXPECT_LT(result.score, 0.5f);
}

TEST(SurfaceDetector, PerformanceUnder15ms) {
    cv::Mat frame = cv::imread('fixtures/clean_metal_01.jpg');
    SurfaceDetector det;
    auto result = det.detect(frame);
    EXPECT_LT(result.duration_ms, 15);
}
```

6.2 Python Tests — pytest

```
# tests/unit/test_vision_bridge.py
import pytest, numpy as np, cv2
from src.vision_bridge import inspect_frame

def test_clean_frame_passes():
    frame = cv2.imread('tests/fixtures/clean_metal_01.jpg')
    result = inspect_frame(frame)
    assert result['pass'] == True
    assert result['confidence'] >= 0.8

def test_scratch_frame_fails():
    frame = cv2.imread('tests/fixtures/scratch_01.jpg')
    result = inspect_frame(frame)
    assert result['pass'] == False
    assert len(result['defects']) >= 1

def test_duration_under_30ms():
    frame = cv2.imread('tests/fixtures/clean_metal_01.jpg')
    result = inspect_frame(frame)
    assert result['duration_ms'] < 30
```

6.3 Required Test Fixtures

File	Content	Expected
clean_metal_01.jpg	No defects, metal surface	PASS
clean_plastic_01.jpg	No defects, plastic surface	PASS
scratch_01.jpg	Single scratch	FAIL

dust_01.jpg	Dust particles	FAIL
color_anomaly_01.jpg	Coating missing / wrong color	FAIL
overexposed_01.jpg	Too bright	FAIL
underexposed_01.jpg	Too dark	FAIL

💡 Take these photos with your UC852 camera from actual factory samples. Real images beat synthetic ones every time.

7. CI/CD PIPELINE

```
# .github/workflows/ci.yml
name: Quality Platform CI
on:
  push:
    branches: [main, develop]

jobs:
  cpp-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Install OpenCV
        run: sudo apt-get install -y libopencv-dev libgtest-dev cmake
      - name: Build C++
        run: cd cpp && cmake -B build && cmake --build build
      - name: Run Google Tests
        run: cd cpp/build && ctest --output-on-failure

  python-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'
      - run: pip install -r requirements.txt pytest pytest-cov
      - run: pytest tests/ -v --cov=src --cov-fail-under=70

  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: pip install flake8 && flake8 src/ tests/
```

8. CLAUDE INTEGRATION

When	What Claude Does	How
Writing any module	Generate C++ header + .cpp + Python bridge	Paste spec into Claude Code terminal
After writing module	Generate Google Test + pytest cases	Prompt: 'Write tests for this interface'
Tests fail	Debug and fix code	Paste error + code into Claude Code
False positive > 5%	Tune thresholds in config.yaml	Prompt: 'FP rate too high, tune these params'
Weekly	Generate shift report from DB stats	claude_helper.py runtime API call
Before PR merge	Code review	Prompt: 'Review this for bugs and improvements'

8.1 claude_helper.py

```
# src/utils/claude_helper.py
import anthropic, base64, cv2, json

client = anthropic.Anthropic()  # ANTHROPIC_API_KEY from env

def classify_defect(frame, region: dict) -> dict:
    x,y,w,h = region['x'],region['y'],region['w'],region['h']
    crop = frame[y:y+h, x:x+w]
    _, buf = cv2.imencode('.jpg', crop)
    img_b64 = base64.b64encode(buf).decode()
    resp = client.messages.create(
        model='claude-sonnet-4-6', max_tokens=200,
        messages=[{'role':'user','content':[

{'type':'image','source':{'type':'base64','media_type':'image/jpeg','data':img_b64}},
            {'type':'text','text':'Classify defect: scratch/dust/color/ok. JSON only:
{type,confidence}'}
        ]}]
    )
    return json.loads(resp.content[0].text)

def shift_report(stats: dict) -> str:
    resp = client.messages.create(
        model='claude-sonnet-4-6', max_tokens=500,
        messages=[{'role':'user','content':f'Write shift quality report for factory
manager. Data: {stats}'}]
    )
    return resp.content[0].text
```

9. GETTING STARTED — DAY 1

Step	Action	Verify
1	Install: Python 3.10, CMake, OpenCV dev libs, GTest	python --version · cmake --version
2	pip install opencv-python fastapi uvicorn sqlalchemy anthropic pytest	No errors
3	Create folder structure (see Section 3)	ls cpp/ src/ tests/
4	Add .env: ANTHROPIC_API_KEY=your_key	cat .env
5	Connect Arducam UC852 via USB	Device Manager shows camera
6	Run camera test: python test_camera.py	1280×800 color window opens
7	Open Claude Code terminal in project folder	claude code .
8	Paste Week 1 prompt from claude_prompts/01_write_module.txt	Claude generates capture.py

First Claude Code Prompt

Paste this into Claude Code terminal on Day 1:

```
I am building an AI quality inspection system for automotive manufacturing.
Camera: Arducam UC852 (OV9782, color, global shutter, USB 2.0)
Stack: C++17 + OpenCV 4.x for vision, Python 3.10 + FastAPI for app layer.
Open standard: GigE Vision / GenICam architecture.

Please generate for Week 1:
1. cpp/include/capture.hpp — camera init and frame grab interface
2. cpp/src/capture.cpp — implementation using OpenCV VideoCapture
3. cpp/tests/test_capture.cpp — Google Test unit tests
4. src/vision_bridge.py — Python wrapper calling C++ binary
5. tests/unit/test_vision_bridge.py — pytest tests

After each file, show me the exact command to build and run tests.
```